

ECS Metrics Observability Pipeline with ADOT, AMP, and Grafana

This document explains the metrics observability pipeline implemented in [examples/fargate](#). It covers the original task, the final architecture, how the Terraform code works, how metrics flow through the system, and how to validate the deployment end to end.

Objective

The task was to configure an end-to-end metrics pipeline for ECS workloads and AWS service metrics using Terraform.

The expected flow was:

```
ECS services / application metrics / AWS service metrics
-> ADOT Collector on ECS
-> Amazon Managed Service for Prometheus using remote_write
-> Grafana dashboards and PromQL queries
```

The implementation uses [terraform-aws-modules](#) for the ECS, ALB, and VPC resources and deploys the observability components as ECS services.

Final Architecture

The final deployed architecture is:

```
Application ECS services
- stig
- windmil
- orchestrator
- grafana

AWS service metrics in CloudWatch
- AWS/ECS
- AWS/ApplicationELB
- AWS/RDS

CloudWatch Exporter ECS service
- Reads CloudWatch metrics
- Exposes them as Prometheus metrics on /metrics

ADOT Collector ECS service
- Discovers ECS scrape targets with ecs_observer
```

- Scrapes Prometheus metrics
- Sends metrics to AMP using prometheusremotewrite with SigV4

Amazon Managed Service for Prometheus

- Stores metrics
- Provides the Prometheus-compatible query API

Grafana ECS service

- Provisions AMP as a Prometheus data source
- Uses SigV4 and the ECS task role to query AMP

Why CloudWatch Exporter Is Used

ADOT Collector image `public.ecr.aws/aws-observability/aws-otel-collector:v0.48.0` does not include an `awsccloudwatch` receiver. When that receiver was attempted, the collector failed to start with:

```
'receivers' unknown type: "awsccloudwatch"
```

Because ECS service metrics, ALB metrics, and RDS metrics are CloudWatch metrics, the implementation uses the Prometheus CloudWatch Exporter as a dedicated ECS service. ADOT then scrapes the exporter like any other Prometheus target.

This keeps ADOT responsible for collection, batching, and remote write to AMP, while CloudWatch Exporter is responsible for converting CloudWatch metrics into Prometheus format.

Terraform Layout

The example is implemented in:

```
examples/fargate/
```

Important files:

```
main.tf
adot-config.yaml.tftpl
variables.tf
terraform.tfvars
versions.tf
README.md
```

Key Resources

VPC

The example creates a VPC using:

```
terraform-aws-modules/vpc/aws
```

It creates public subnets for the ALB and private subnets for ECS tasks.

ALB

The example creates an Application Load Balancer using:

```
terraform-aws-modules/alb/aws
```

The ALB exposes:

stig	port 80
windmil	port 8081
orchestrator	port 8082
grafana	port 3000

ECS Cluster and Services

The example uses the root ECS module:

```
module "ecs_cluster" {  
  source = "../.."
```

It creates these ECS services:

```
stig  
windmil  
orchestrator  
grafana  
adot  
cloudwatch-exporter
```

ADOT Collector

The ADOT service runs:

```
public.ecr.aws/aws-observability/aws-otel-collector:v0.48.0
```

Its configuration is rendered from:

```
examples/fargate/adot-config.yaml.tftpl
```

The rendered config is stored in SSM Parameter Store at:

```
/clouddrove-fargate/adot/config
```

The ADOT task receives the config through the `AOT_CONFIG_CONTENT` secret.

The ADOT service has a Terraform service trigger:

```
triggers = {  
  adot_config_sha = sha256(local.adot_config_content)  
}
```

This causes the ECS service to redeploy when the collector config changes.

CloudWatch Exporter

The CloudWatch exporter service runs:

```
prom/cloudwatch-exporter:v0.16.0
```

The exporter listens on:

```
9106
```

It exposes metrics at:

```
/metrics
```

The exporter config is generated with `yamllencode` in `main.tf` and stored in SSM Parameter Store at:

```
/clouddrove-fargate/cloudwatch-exporter/config
```

A small `busybox` sidecar writes the SSM-provided config into:

```
/config/config.yml
```

The exporter container mounts that config path and starts with it.

The service has Docker labels used by ADOT ECS discovery:

```
ECS_PROMETHEUS_EXPORTER_PORT = "9106"  
ECS_PROMETHEUS_EXPORTER_PATH = "/metrics"  
ECS_PROMETHEUS_JOB_NAME      = "cloudwatch-exporter"
```

Grafana

The Grafana service runs:

```
grafana/grafana:latest
```

Grafana metrics are enabled with:

```
GF_METRICS_ENABLED = "true"
```

SigV4 support is enabled with:

```
GF_AUTH_SIGV4_AUTH_ENABLED = "true"
```

The AMP data source is provisioned automatically at container startup. This was done because the Grafana UI did not show the SigV4 authentication option in the manual data source form.

The rendered data source config is stored in SSM Parameter Store at:

```
/clouddrove-fargate/grafana/datasource
```

The Grafana task receives it as `GRAFANA_DATASOURCE_CONFIG` and writes it to:

```
/etc/grafana/provisioning/datasources/amp.yaml
```

The provisioned data source is named:

```
AMP
```

It points to the AMP query endpoint, not the remote write endpoint.

ADOT Configuration

The collector configuration uses:

```
extensions:
  sigv4auth:
    service: aps
    region: ${region}

  ecs_observer:
    cluster_name: ${cluster_name}
    cluster_region: ${region}
    refresh_interval: 60s
    result_file: /etc/ecs_sd_targets.yaml
```

`ecs_observer` discovers ECS tasks that have the expected Docker labels and writes Prometheus file service discovery targets to:

```
/etc/ecs_sd_targets.yaml
```

The Prometheus receiver reads that file:

```
receivers:
  prometheus:
    config:
      scrape_configs:
        - job_name: ecs-services
          file_sd_configs:
            - files:
                - /etc/ecs_sd_targets.yaml
```

The exporter uses AMP remote write with SigV4:

```
exporters:
  prometheusremotewrite:
    endpoint: ${remote_write_endpoint}
    auth:
      authenticator: sigv4auth
```

IAM Permissions

ADOT Task Role

ADOT needs permission to remote write metrics to AMP:

```
aps:RemoteWrite
```

It also needs ECS discovery permissions:

```
ecs:ListTasks
ecs:ListServices
ecs:DescribeServices
ecs:DescribeTasks
ecs:DescribeTaskDefinition
ecs:DescribeContainerInstances
```

CloudWatch Exporter Task Role

CloudWatch Exporter needs:

```
cloudwatch:GetMetricData
cloudwatch:GetMetricStatistics
```

```
cloudwatch:ListMetrics
tag:GetResources
```

Grafana Task Role

Grafana needs AMP query permissions:

```
aps:QueryMetrics
aps:GetLabels
aps:GetSeries
aps:GetMetricMetadata
```

Metrics Collected

The CloudWatch exporter is configured to collect:

ECS

```
AWS/ECS CPUUtilization
AWS/ECS MemoryUtilization
```

Dimensions:

```
ClusterName
ServiceName
```

Application Load Balancer

```
AWS/ApplicationELB RequestCount
AWS/ApplicationELB TargetResponseTime
AWS/ApplicationELB HTTPCode_Target_2XX_Count
AWS/ApplicationELB HTTPCode_Target_4XX_Count
AWS/ApplicationELB HTTPCode_Target_5XX_Count
```

Dimension:

```
LoadBalancer
```

RDS

```
AWS/RDS CPUUtilization
AWS/RDS DatabaseConnections
AWS/RDS FreeableMemory
```

Dimension:

```
DBInstanceIdentifier
```

RDS queries return data only if RDS instances exist in the configured region.

Deployment Steps

1. Create or identify an AMP workspace.
2. Update the AMP workspace ID in:

```
examples/fargate/terraform.tfvars
```

Example:

```
amp_workspace_id = "ws-xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx"
```

3. Update the AMP remote write endpoint in `main.tf` if using another workspace:

```
amp_remote_write_endpoint = "https://aps-workspaces.eu-west-1.amazonaws.com/workspaces/ws-xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx/api/v1/remote_write"
```

4. Initialize Terraform:

```
terraform -chdir=examples/fargate init
```

5. Review the plan:

```
terraform -chdir=examples/fargate plan
```

6. Apply:

```
terraform -chdir=examples/fargate apply
```

Validation

Check ECS Services

```
aws ecs describe-services \
  --region eu-west-1 \
  --cluster clouddrove-fargate \
  --services adot cloudwatch-exporter grafana
```

Each service should have the expected desired and running counts.

Check ADOT Logs

```
aws logs tail /ecs/cloudrove-fargate/adot \  
  --region eu-west-1 \  
  --follow
```

Healthy startup logs include:

```
Starting ECSDiscovery  
Starting scrape manager  
starting prometheus remote write exporter  
Everything is ready. Begin running and processing data.
```

Open Grafana

Get the ALB DNS name:

```
aws elbv2 describe-load-balancers \  
  --region eu-west-1 \  
  --names cloudrove-fargate \  
  --query 'LoadBalancers[0].DNSName' \  
  --output text
```

Open:

```
http://<alb-dns-name>:3000
```

The default Grafana credentials are:

```
username: admin  
password: admin
```

Change the password when prompted.

Confirm AMP Data Source

In Grafana, go to:

```
Connections -> Data sources
```

You should see:

```
AMP
```

The data source is provisioned automatically and uses SigV4.

Run PromQL Queries

In Grafana:

```
Explore -> AMP -> Code
```

Run:

```
up
```

Expected result:

```
1
```

for `grafana` and `cloudwatch-exporter` scrape targets.

Run:

```
cloudwatch_exporter_scrape_error
```

Expected result:

```
0
```

Run:

```
aws_ecs_cpuutilization_average
```

Expected result:

```
ECS CPU metrics by service
```

Run:

```
aws_applicationelb_request_count_sum
```

Expected result:

```
ALB request count metrics
```

Useful Dashboard Queries

ECS CPU utilization by service:

```
avg by (service_name) (aws_ecs_cpuutilization_average)
```

ALB request count:

```
sum(aws_applicationelb_request_count_sum)
```

ALB target response time:

```
avg(aws_applicationelb_target_response_time_average)
```

ALB 5xx responses:

```
sum(aws_applicationelb_httpcode_target_5_xx_count_sum)
```

CloudWatch exporter scrape health:

```
cloudwatch_exporter_scrape_error
```

Prometheus target health:

```
up
```

Troubleshooting

ADOT error: no such file or directory for `/tmp/`

The collector originally attempted to watch `/tmp/ecs_sd_targets.yaml`. The final config uses:

```
/etc/ecs_sd_targets.yaml
```

If the task still references `/tmp`, the running ECS task is using an older SSM config. Force a new deployment:

```
aws ecs update-service \  
  --region eu-west-1 \  
  --cluster clouddrove-fargate \  
  --service adot \  
  --force-new-deployment
```

ADOT error: unknown receiver `awsc loudwatch`

The ADOT image used by this example does not support the `awsc loudwatch` receiver. The final solution uses CloudWatch Exporter instead.

Grafana returns 403 for AMP

AMP requires SigV4 signed requests. Using "No Authentication" in the Grafana UI will return:

This example provisions the `AMP` data source automatically with SigV4 enabled. Use the provisioned `AMP` data source.

`up` is empty

Check that ADOT discovered scrape targets and that the target services are running:

```
aws logs tail /ecs/clouddrove-fargate/adot \  
  --region eu-west-1 \  
  --since 10m
```

Also verify the target services are running:

```
aws ecs describe-services \  
  --region eu-west-1 \  
  --cluster clouddrove-fargate \  
  --services adot cloudwatch-exporter grafana
```

ECS memory metric is empty

`aws_ecs_memoryutilization_average` may be empty depending on whether CloudWatch has emitted that metric for the selected service and time range. If `aws_ecs_cpuutilization_average` and `aws_applicationlb_request_count_sum` return data, the pipeline is working.

RDS metrics are empty

RDS queries return data only when RDS instances exist in the configured region and emit metrics during the selected time range.

Cleanup

To remove the resources:

```
terraform -chdir=examples/fargate destroy
```

The example creates billable resources, including ECS services, NAT gateway, ALB, CloudWatch logs, and AMP usage.